

```

#include <omp.h>
#include <stdio.h>

// Buffer de 1024 inteiros
int buffer[1024];
int insert = 0;
int consume = 0;
int count = 0;

// Insere um item no buffer circular. A implementação está passível de overflow caso
// sera inserido mais de 1024 itens sem ninguém consumir.
void producer(int arg) {
#pragma omp critical
{
    printf("Inserindo %d na pos %d\n", arg, insert);
    buffer[insert] = arg;
    // Avança o ponteiro de itens inseridos.
    insert = (insert + 1) % 1024;
    // Adiciona mais um na contagem, indicando que tem algo pra ser consumido.
    count++;
}
}

// Consome um item do buffer se tiver algo para consumir, caso contrário retorna -1.
int consumer() {
#pragma omp critical
{
    int v = -1;
    {
        // Só consome se tiver alguém para ser consumido.
        if (count > 0) {
            v = buffer[consume];
            // Avança o ponteiro de itens consumidos.
            consume = (consume + 1) % 1024;
            // Indica que 1 item foi consumido.
            count--;
            printf("Buscando dado %d na pos %d\n", v, consume);
        }
    }
    // Retorna -1 se não tinha como ler
    return v;
}
}

int main() {
    // Cria região paralela
#pragma omp parallel
    {
        // Bota 1 thread para inserir
#pragma omp master
        {
            for (int i = 0; i < 10; i++) {
                producer(3 * i + 15 * 4 / 3);
            }
        }

        // Bota outra thread para consumir.
#pragma omp single

```

```
{  
    for (int i = 0; i < 10; i++) {  
        int val = -1;  
        while (val == -1) {  
            val = consumer();  
        }  
    }  
}  
}  
return 0;  
}
```